E-Privacy

Implementation Report

A privacy preserving analytics platform that computes statistics on encrypted data, without ever revealing the original values to the server that does the computing.

Dilmith D. L. S.	220123C
Hansana O. G. V.	220211U
Vidyananda H. K. H.	220666R

09/10/2026

Contents

1 Overview 3

2 System architecture 4

3 Technology stack 6

4 Core protocol and data flow 7

5 Database schema 11

6 Code listings 13

7 User interface design 20

8 Runtime screen captures 21

9 Testing and measured results 28

10 Limitations and future work 30

1 Overview

E-Privacy lets a data owner hand computation to a server it does not trust. Before any record leaves the owner, every numeric measure is encrypted with the Paillier cryptosystem. Paillier is additively homomorphic: multiplying two ciphertexts modulo n^2 yields an encryption of the sum of the two plaintexts. The cloud can therefore add up thousands of encrypted values and return an encrypted total that only the owner, who holds the private key, can read.

Encryption alone does not stop a dishonest server from dropping a record, inventing one or reporting the wrong total. The implementation adds the pieces a real deployment needs on top of the cipher:

- **Access control.** Three cloud roles (owner service, auditor, operator) behind a policy gateway that writes every request and its outcome to a query log.
- **Receipts.** The cloud signs a Merkle root over every record it stored, which the owner compares with a root it computed itself.
- **A public bulletin board.** Every campaign, receipt, encrypted result and decryption proof is appended to a signed hash chain that anyone with read access can replay.
- **Result verification.** The owner accepts a result only after five independent checks pass: the result carries the cloud's signature, it matches the copy on the public board, it was computed over exactly the records in the upload receipt, the record count is unchanged, and every decrypted sum equals the owner's own recomputation. The owner then publishes a proof that lets any auditor confirm the decryption without the private key.
- **Small group suppression.** Groups with fewer than k records (10 by default) are never released.

The system is exercised with a synthetic test dataset of 540,000 rows and 26 columns, produced by a seeded generator.

2 System architecture

The system is split along a single trust boundary. Everything that can see plaintext or a private key sits on the owner side; everything on the cloud side works only with public keys, ciphertext and coarse labels. A third party, the auditor, needs nothing beyond read access to public data.

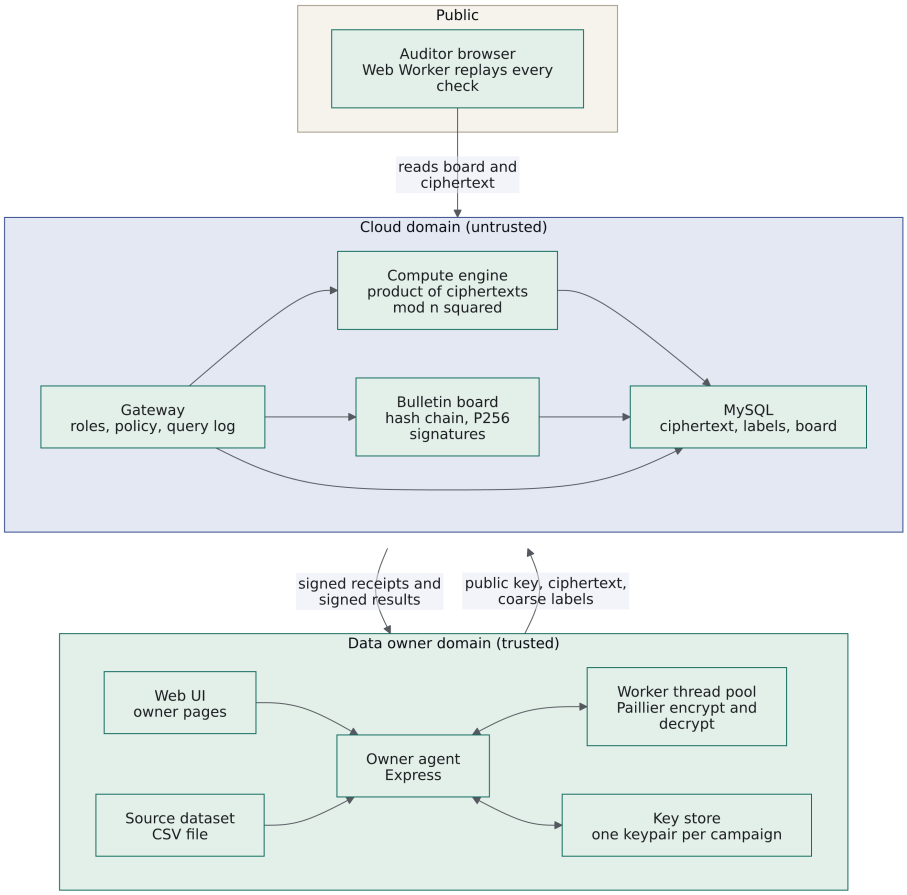


Figure 1. Components and the trust boundary. Arrows show the only data that crosses from one domain to the other.

2.1 Components

Component	Domain	Responsibility
Owner agent (owner/)	Trusted	Generates one Paillier keypair per campaign, encrypts on a worker pool, uploads, checks receipts, verifies and decrypts results.
Cloud gateway (cloud/)	Untrusted	Authenticates sessions, enforces role policy, stores ciphertext, issues signed receipts, runs homomorphic queries, serves monitoring data.
Bulletin board (cloud/src/board.js)	Untrusted, verifiable	Append only log. Each entry is SHA-256 chained to the previous one and signed with the cloud's P-256 key.
Shared protocol (shared/)	Both and the browser	Record layout, leaf and Merkle hashing, board chain, signatures and the public decryption check. One module, imported by all three parties.
Web UI (web/)	Both	Vue 3 single page app. Owner pages talk to the owner agent; auditor and operator pages talk to the cloud.
Auditor worker (web/src/audit.worker.js)	Public	Runs the four universal checks in a Web Worker using only public data.
MySQL 8.4 (db/)	Untrusted	Six tables; no identifiers, plaintext measures or private keys.

2.2 Deployment

All four services run under Docker Compose. Two networks enforce the boundary at the network level: the database sits on *cloud_net* only, which the owner container is not attached to. The owner therefore has no route to the database and no database credentials; it reaches the cloud only through the HTTP API. Each container receives only its own environment variables.

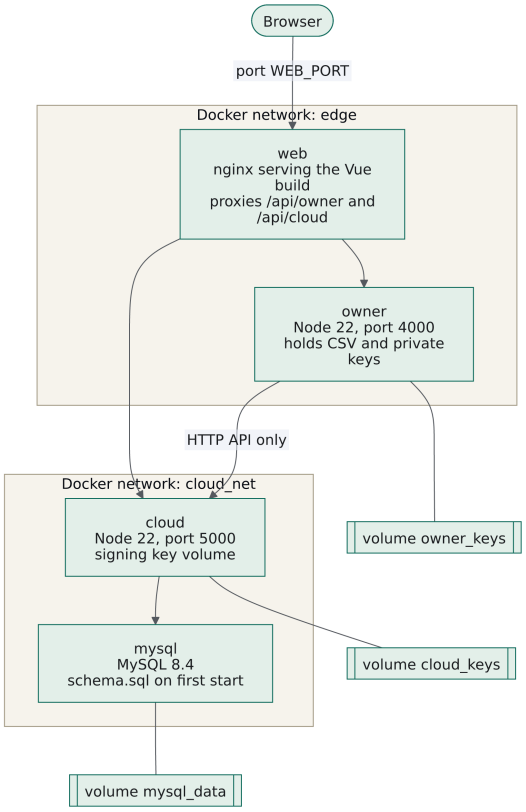


Figure 2. Containers, networks and volumes as defined in docker-compose.yml.

3 Technology stack

Layer	Choice	Reason
Homomorphic encryption	Paillier via paillier-bigint 3.4	Additive only, which is exactly what sums, means, rates and variances need. Fully homomorphic schemes were benchmarked at roughly 600 times the storage.
Hashing and signatures	@noble/hashes SHA-256, @noble/curves P-256	Audited, dependency free, and identical in Node and in the browser.
Services	Node.js 22 with Express	Native BigInt and worker_threads for the CPU bound cryptography.
Database	MySQL 8.4	Relational tables, LONGBLOB for packed ciphertext, information_schema for storage metrics.
Frontend	Vue 3, Vue Router, ECharts, built by Vite	Small runtime, lazy loaded routes, charts with accessible labels.
Serving	nginx	Serves the static build and proxies /api/owner and /api/cloud.
Packaging	Docker Compose v2	One command brings up the whole system; a development override adds hot reload.
Password storage	scrypt (Node crypto)	Memory hard key derivation with a random salt per user.

4 Core protocol and data flow

4.1 Creating a campaign

A campaign is one encrypted upload with its own keypair. A fresh key per campaign limits what a leaked key could expose. The owner agent runs the steps below as a background job and reports its progress to the UI. Encryption is the slow step; it is spread across every core except one.

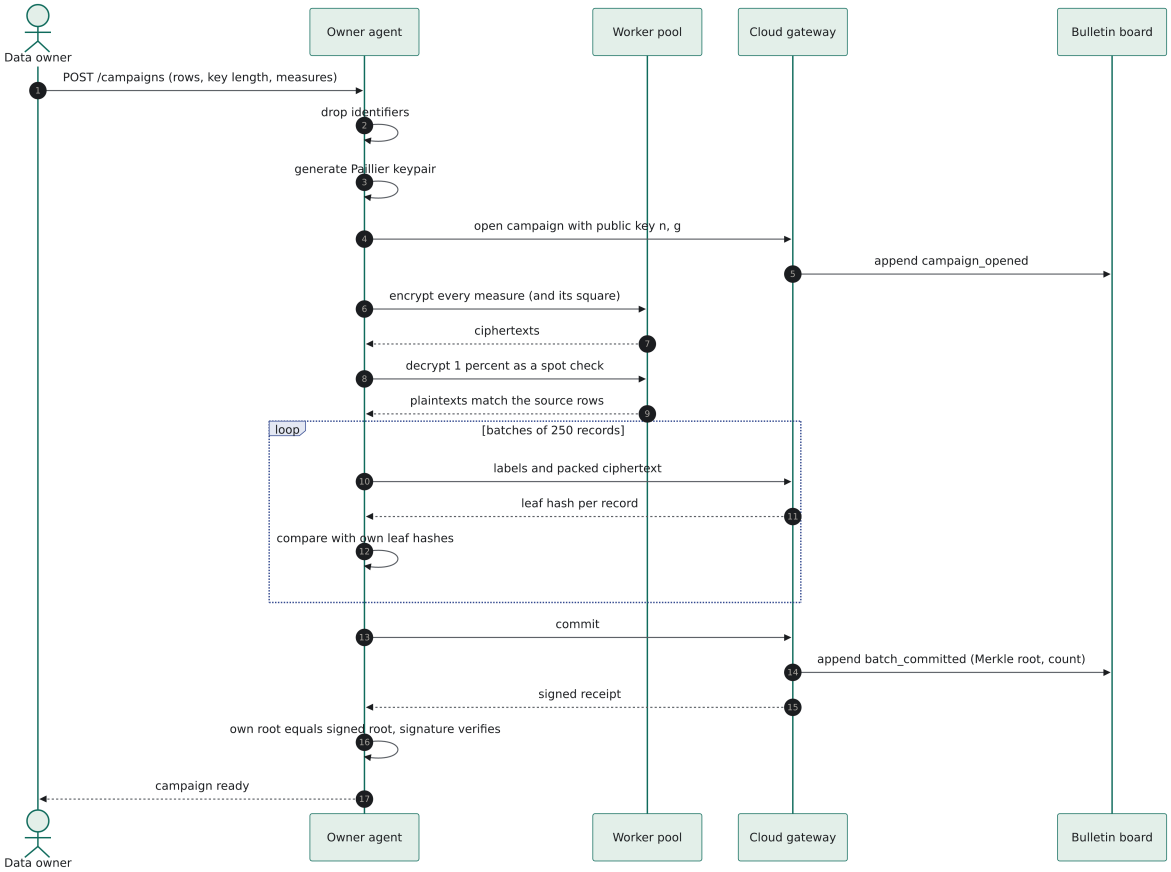


Figure 3. Campaign creation, from key generation to a verified receipt.

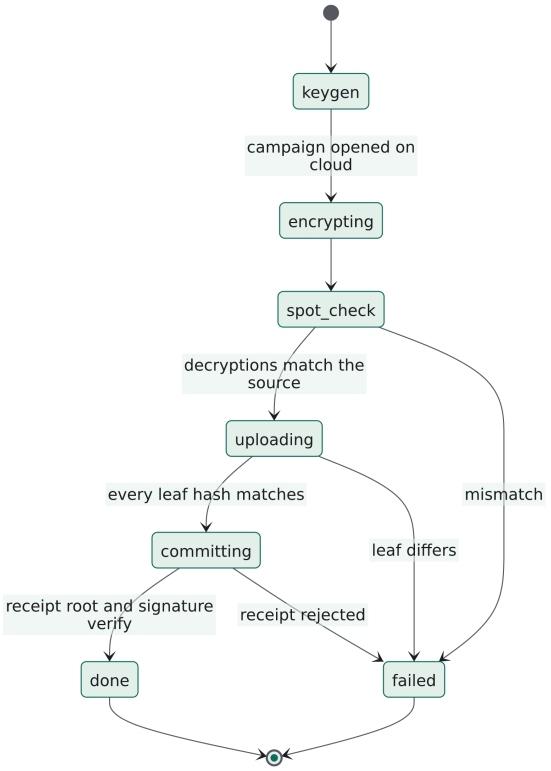


Figure 4. Phases of the campaign job. Any failed check stops the job before the next step.

4.2 Record layout and receipts

Each stored record carries its seven labels in clear and its ciphertexts packed into one binary blob: each chosen measure, then the encryption of its square when spread statistics are wanted. At 2048 bits every ciphertext is exactly 512 bytes. The leaf hash binds the labels to the ciphertext, which stops the cloud from quietly moving a record into a different group. Leaves and inner nodes use distinct prefixes (0 and 1), and an odd node is carried up rather than duplicated, which avoids the duplicate leaf ambiguity of some Merkle designs.

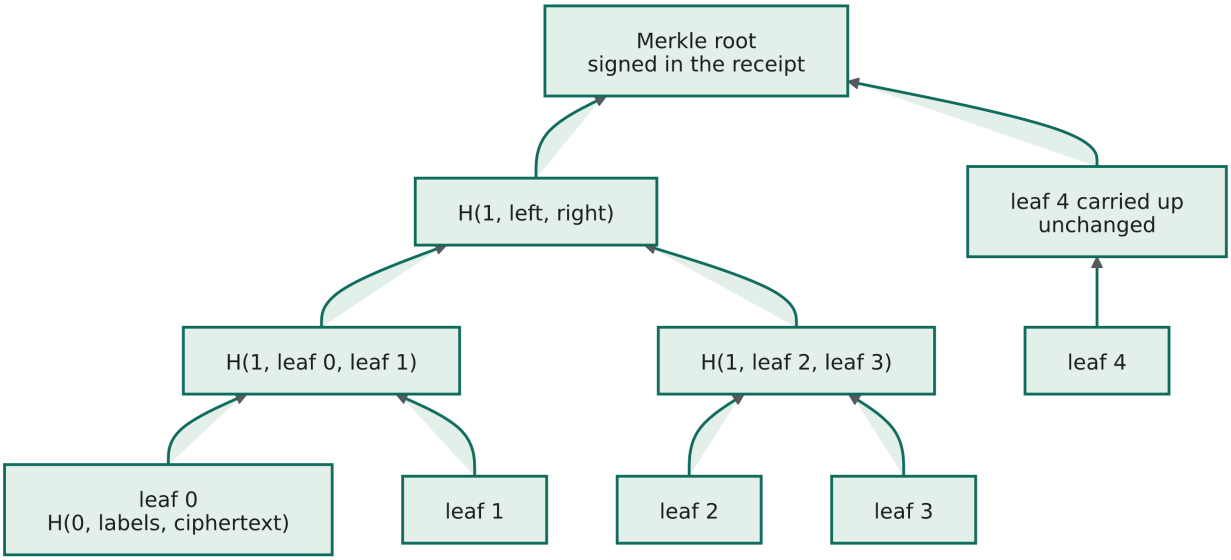


Figure 5. Merkle tree over five records. The signed root becomes the owner's receipt.

4.3 Running a query and verifying it

The cloud groups records by at most two labels and multiplies the ciphertexts of each group modulo n^2. Groups smaller than k are withheld. The signed result also carries the Merkle root and size of the input set it actually used. The owner accepts the result only when all five checks hold: the signature verifies against the key pinned on first contact, the returned entry is identical to the copy on the board, the input root equals the receipt, the record count equals the upload, and every decrypted sum equals a recomputation from the owner's own plaintext.

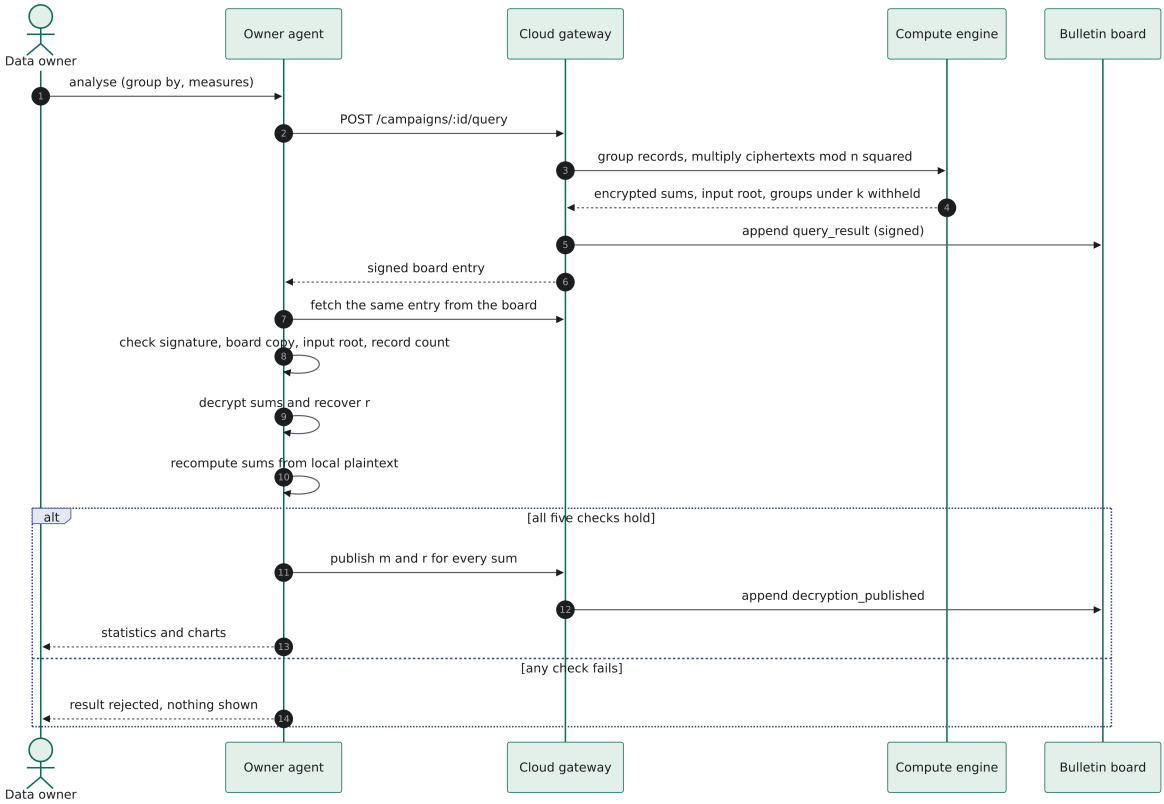


Figure 6. Query, verification and publication of the decryption proof.

Mean and standard deviation come from exact integer sums. With $S = \Sigma x$, $Q = \Sigma x^2$ and group size n, the variance is $(nQ - S^2) / n^2$. Values with decimals are scaled to integers before encryption (for example by 100 for currency).

4.4 Universal audit

When the owner decrypts a sum it also recovers the Paillier randomness r and publishes the pair (m, r). Anyone holding the public key can then confirm that $g^m \cdot r^n \equiv C \pmod{n^2}$. The auditor page replays the whole board in a Web Worker and runs four checks that together cover every step of the pipeline.

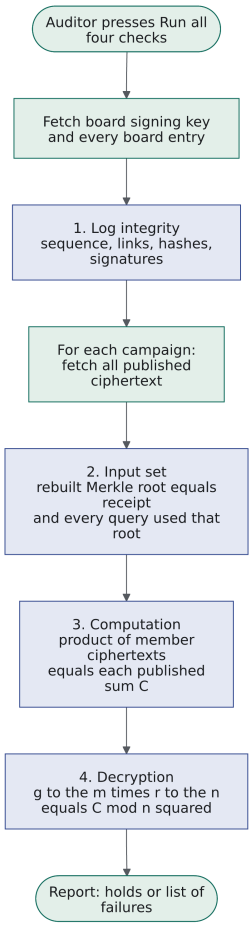


Figure 7. The four checks run by the auditor. None of them needs a private key.

5 Database schema

The database belongs to the cloud and holds six tables. It starts empty apart from three service accounts; records arrive only as ciphertext, through a campaign.

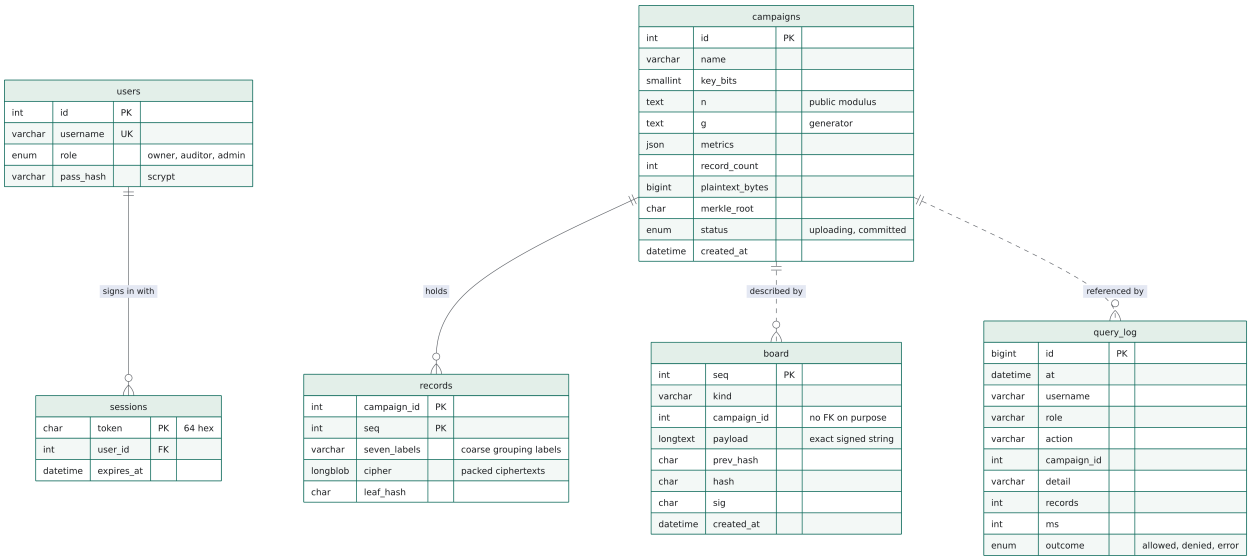


Figure 8. Entity relationship diagram. Dotted lines are logical links without a foreign key.

Table	Holds	Design note
users	username, role, scrypt hash	Synced from the environment at every cloud start, which lets a changed password take effect without touching the database.
sessions	random 64 hex token, user, expiry	Deleted on sign out and cascaded when a user goes.
campaigns	key length, public key n and g, measure layout, Merkle root, status	Status moves one way, from uploading to committed. No upload is accepted after commit.
records	seven labels, packed ciphertext, leaf hash	Labels are real columns because they are the exact fields bound into each leaf. Binary storage halves the size compared with hex text.
board	sequence, kind, payload, previous hash, hash, signature	The payload is LONGTEXT, not JSON: MySQL would reorder JSON keys and break every signature. The sequence is the primary key, which makes a fork of the chain impossible.
query_log	time, user, role, action, records, duration, outcome	Written for every authenticated request and every sign in attempt, allowed or denied.

A full dump of this database yields ciphertext, public keys, coarse labels, the aggregates the owner chose to publish and the board. It contains no identifiers, no exact values and no key material.

Listing 1. db/schema.sql, complete

```
-- Cloud-side store. Holds ciphertext, coarse grouping dimensions and the
-- bulletin board. No identifier or plaintext measure ever lands here.

CREATE TABLE users (
  id        INT AUTO_INCREMENT PRIMARY KEY,
  username  VARCHAR(64)  NOT NULL UNIQUE,
  role      ENUM('owner', 'auditor', 'admin') NOT NULL,
  pass_hash VARCHAR(255) NOT NULL
);

CREATE TABLE sessions (
  token      CHAR(64) PRIMARY KEY,
  user_id    INT      NOT NULL,
  expires_at DATETIME NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
```

```
);

CREATE TABLE campaigns (
  id           INT AUTO_INCREMENT PRIMARY KEY,
  name         VARCHAR(120) NOT NULL,
  key_bits     SMALLINT     NOT NULL,
  n            TEXT         NOT NULL,          -- public modulus, hex
  g            TEXT         NOT NULL,          -- generator, hex
  metrics      JSON         NOT NULL,          -- [{name, scale, squared}]
  record_count INT          NOT NULL DEFAULT 0,
  plaintext_bytes BIGINT    NOT NULL DEFAULT 0,          -- size of the source rows, reported by the owner
  merkle_root  CHAR(64)     NULL,
  status       ENUM('uploading', 'committed') NOT NULL DEFAULT 'uploading',
  created_at   DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE records (
  campaign_id     INT         NOT NULL,
  seq             INT         NOT NULL,
  department      VARCHAR(32) NOT NULL,
  ward_type       VARCHAR(16) NOT NULL,
  admission_type  VARCHAR(16) NOT NULL,
  insurance_type  VARCHAR(24) NOT NULL,
  admission_month CHAR(7)     NOT NULL,
  sex             CHAR(1)     NOT NULL,
  age_band        VARCHAR(8)  NOT NULL,
  cipher          LONGBLOB    NOT NULL,         -- fixed-width ciphertexts, concatenated
  leaf_hash       CHAR(64)    NOT NULL,
  PRIMARY KEY (campaign_id, seq),
  FOREIGN KEY (campaign_id) REFERENCES campaigns(id) ON DELETE CASCADE
);

-- Append-only, hash-chained, signed. payload is stored as the exact string
-- that was hashed so key order can never drift.
CREATE TABLE board (
  seq         INT PRIMARY KEY,
  kind        VARCHAR(32) NOT NULL,
  campaign_id INT         NULL,
  payload     LONGTEXT    NOT NULL,
  prev_hash   CHAR(64)    NOT NULL,
  hash        CHAR(64)    NOT NULL,
  sig         CHAR(128)   NOT NULL,
  created_at  DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)
);

CREATE TABLE query_log (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  at          DATETIME(3)  NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  username    VARCHAR(64)  NULL,
  role        VARCHAR(16)  NULL,
  action      VARCHAR(48)  NOT NULL,
  campaign_id INT          NULL,
  detail      VARCHAR(255) NULL,
  records     INT          NULL,
  ms          INT          NULL,
  outcome     ENUM('allowed', 'denied', 'error') NOT NULL
);
CREATE INDEX query_log_at ON query_log (at);
```

6 Code listings

The listings below are taken directly from the repository when this report is generated. They cover the shared protocol, the owner agent, the cloud gateway and the auditor.

6.1 Shared protocol

Listing 2. shared/protocol.js: leaf hashing and Merkle proofs

```
// Leaf binds the grouping dimensions to the ciphertexts, so the cloud can't
// quietly move a record into another group.
export function leafHash(dims, cipherBytes) {
  const label = utf8ToBytes(DIMENSIONS.map((d) => dims[d]).join('|'));
  return bytesToHex(sha256(concatBytes(Uint8Array.of(0), label, Uint8Array.of(0), cipherBytes)));
}

const node = (l, r) => bytesToHex(sha256(concatBytes(Uint8Array.of(1), hexToBytes(l), hexToBytes(r))));

// An odd node is carried up unchanged rather than duplicated, which avoids the
// duplicate-leaf ambiguity of Bitcoin-style trees.
function levels(leaves) {
  const out = [leaves];
  while (out[out.length - 1].length > 1) {
    const prev = out[out.length - 1];
    const next = [];
    for (let i = 0; i < prev.length; i += 2) next.push(i + 1 < prev.length ? node(prev[i], prev[i + 1]) : pr ↵
        ev[i]);
    out.push(next);
  }
  return out;
}

export function merkleRoot(leaves) {
  if (!leaves.length) return sha256hex('');
  const lv = levels(leaves);
  return lv[lv.length - 1][0];
}

export function merkleProof(leaves, index) {
  const proof = [];
  for (const level of levels(leaves).slice(0, -1)) {
    const sib = index ^ 1;
    if (sib < level.length) proof.push({ side: sib < index ? 'L' : 'R', hash: level[sib] });
    index >>= 1;
  }
  return proof;
}

export function verifyProof(leaf, proof, root) {
  let h = leaf;
  for (const { side, hash } of proof) h = side === 'L' ? node(hash, h) : node(h, hash);
  return h === root;
}
```

Listing 3. shared/protocol.js: board chain verification and the public decryption check

```
// Returns null when the whole chain holds, else { seq, reason } for the first break.
export function verifyChain(entries, pubHex) {
  let prev = GENESIS;
  for (const [i, e] of entries.entries()) {
    if (e.seq !== i + 1) return { seq: e.seq, reason: 'sequence gap' };
    if (e.prev_hash !== prev) return { seq: e.seq, reason: 'broken link to previous entry' };
    if (entryHash(e) !== e.hash) return { seq: e.seq, reason: 'entry content does not match its hash' };
    if (!verifySig(e.hash, e.sig, pubHex)) return { seq: e.seq, reason: 'bad signature' };
    prev = e.hash;
  }
  return null;
}

export function aggregate(ciphers, n2) {
  let acc = 1n;
  for (const c of ciphers) acc = (acc * c) % n2;
  return acc;
}

// Anyone holding the public key can confirm that C decrypts to m, given the
// randomness r the owner recovered: g^m * r^n == C (mod n^2).
```

```
export function verifyDecryption({ n, g, C, m, r }) {
  const n2 = n * n;
  return (modPow(g, m, n2) * modPow(r, n, n2)) % n2 === C % n2;
}
```

6.2 Owner agent

Listing 4. owner/src/worker.js, complete: Paillier work off the main thread

```
// Runs inside the trusted domain only. Paillier work is CPU-bound, so it is
// spread across worker threads (see pool.js).
import { parentPort } from 'node:worker_threads';
import { PublicKey, PrivateKey } from 'paillier-bigint';

const big = (hex) => BigInt('0x' + hex);

parentPort.on('message', ({ id, type, key, items }) => {
  try {
    const pub = new PublicKey(big(key.n), big(key.g));
    let result;
    if (type === 'encrypt') {
      // items: array of rows, each an array of decimal plaintext strings
      result = items.map((row) => row.map((m) => pub.encrypt(BigInt(m)).toString(16)));
    } else if (type === 'decrypt') {
      const priv = new PrivateKey(big(key.lambda), big(key.mu), pub, big(key.p), big(key.q));
      // items: hex ciphertexts; r is recovered so the result can be proven publicly
      result = items.map((c) => ({ m: priv.decrypt(big(c)).toString(16), r: priv.getRandomFactor(big(c)).toS ↵
        tring(16) }));
    } else {
      throw new Error(`unknown task ${type}`);
    }
    parentPort.postMessage({ id, result });
  } catch (err) {
    parentPort.postMessage({ id, error: err.message });
  }
});
```

Listing 5. owner/src/campaign.js: spot check, upload and receipt verification

```
    // Spot-check before anything leaves the building (Verifiability, "encrypted as intended").
    job.phase = 'spot-check';
    const checks = Math.max(5, Math.ceil(picked.length * 0.01));
    const sampleIdx = Array.from({ length: checks }, () => Math.floor(Math.random() * picked.length));
    const decrypted = await pool.map('decrypt', key, sampleIdx.map((i) => ciphers[i][0]), 4);
    const mismatch = sampleIdx.findIndex((i, j) => BigInt('0x' + decrypted[j].m) !== BigInt(plain[i][0]));
    if (mismatch >= 0) throw new Error(`Spot-check failed on sampled record ${sampleIdx[mismatch]}; nothing  ↵
        was uploaded`);
    note(`Spot-check: decrypted ${checks} random ciphertexts, all match the source rows`);

    job.phase = 'uploading';
    job.done = 0;
    const width = cipherWidth(keyBits);
    const records = picked.map((p, seq) => {
      const blob = new Uint8Array(columns.length * width);
      ciphers[seq].forEach((c, j) => blob.set(bigintToBytes(BigInt('0x' + c), width), j * width));
      return { seq, dims: dims(p.row), blob };
    });
    const leaves = records.map((r) => leafHash(r.dims, r.blob));
    for (let i = 0; i < records.length; i += UPLOAD_BATCH) {
      const batch = records.slice(i, i + UPLOAD_BATCH);
      const res = await cloud('POST', `/campaigns/${id}/records`, {
        records: batch.map((r) => ({ seq: r.seq, dims: r.dims, cipher: Buffer.from(r.blob).toString('base64' ↵
        ) })),
      });
      const off = res.leaves.findIndex((l, j) => l !== leaves[i + j]);
      if (off >= 0) throw new Error(`Cloud hashed record ${i + off} differently from us`);
      job.done += batch.length;
    }
    note(`Uploaded ${records.length.toLocaleString()} encrypted records; every stored leaf hash matched ours ↵
        `);

    job.phase = 'committing';
    const receipt = await cloud('POST', `/campaigns/${id}/commit`);
    const localRoot = merkleRoot(leaves);
    if (receipt.root !== localRoot || receipt.count !== records.length) throw new Error('Signed receipt root ↵
         does not match our own Merkle root');
    if (entryHash(receipt.entry) !== receipt.entry.hash || !verifySig(receipt.entry.hash, receipt.entry.sig, ↵
         boardKey)) {
      throw new Error('Receipt signature did not verify against the pinned board key');
```

```
    }
    Object.assign(state, { leaves, receipt: { root: localRoot, count: records.length, seq: receipt.entry.seq
        , hash: receipt.entry.hash, sig: receipt.entry.sig } });
    await writeState(state);
    note(`Receipt #${receipt.entry.seq} verified: root ${localRoot.slice(0, 16)}… signed by the cloud`);

    job.phase = 'done';
```

Listing 6. owner/src/campaign.js: the five acceptance checks (excerpt)

```
  // Individual verifiability: is this the signed answer over *our* records?
  const onBoard = await cloud('GET', `/board/${entry.seq}`);
  const checks = [
    { id: 'signature', label: 'Result entry is signed by the pinned cloud key', ok: entryHash(entry) === ent
        ry.hash && verifySig(entry.hash, entry.sig, state.boardKey) },
    { id: 'published', label: 'Returned result is identical to the one on the board', ok: onBoard.hash === e
        ntry.hash && onBoard.payload === entry.payload },
    { id: 'root', label: 'Input set root equals our upload receipt', ok: payload.root === state.receipt.root
        , detail: `${payload.root.slice(0, 12)}… vs ${state.receipt.root.slice(0, 12)}…` },
    { id: 'count', label: 'Record count equals what we uploaded', ok: payload.total === state.count, detail:
         `${payload.total} vs ${state.count}` },
  ];
  // ... decrypt, recompute from plaintext ...
  const mismatches = accuracy.filter((a) => a.diff !== '0').length;
  checks.push({ id: 'plaintext', label: 'Every decrypted sum equals our own plaintext recomputation', ok: mi
        smatches === 0, detail: `${mismatches} of ${accuracy.length} differ` });

  const accepted = checks.every((c) => c.ok);
  let proofSeq = null;
  if (accepted) {
    const proof = await cloud('POST', `/campaigns/${id}/decryptions`, {
      query_seq: entry.seq,
      values: flat.map((f, i) => ({ group: f.gi, column: f.col, m: opened[i].m, r: opened[i].r })),
    });
    proofSeq = proof.seq;
  }
```

Listing 7. owner/src/campaign.js: mean and standard deviation from encrypted sums

```
// mean and population s.d. from exact integer sums: var * n^2 = n * sum(x^2) - sum(x)^2
function stats(g, cols) {
  const out = {};
  const n = BigInt(g.n);
  for (const col of cols) {
    if (col.includes('^')) continue;
    const { scale, flag } = METRIC[col];
    const s = g.sums[col];
    const entry = { sum: Number(s) / scale, mean: Number(s) / scale / g.n };
    const s2 = g.sums[`${col}^2`];
    if (s2 !== undefined) entry.sd = Math.sqrt(Number(n * s2 - s * s)) / g.n / scale;
    else if (flag) entry.sd = Math.sqrt(entry.mean * (1 - entry.mean));
    out[col] = entry;
  }
  return out;
}
```

6.3 Cloud gateway

Listing 8. cloud/src/compute.js, complete: the homomorphic query engine

```
// Homomorphic compute engine (FR-3). Sums are products of ciphertexts mod n^2;
// no key material other than the public modulus is ever available here.
import { DIMENSIONS, cipherColumns, cipherWidth, splitCipher, aggregate, merkleRoot } from '@e-privacy/share
      d';
import { all } from './db.js';

const K_MIN = Number(process.env.K_MIN || 10);

// Demonstration switch for the Verifiability report: 'drop' leaves one record
// out of the input set, 'tamper' doubles one ciphertext. Admin-only.
export const fault = { mode: 'off' };

export async function runQuery(campaign, { groupBy = [], metrics = [] }) {
  const bad = groupBy.find((d) => !DIMENSIONS.includes(d)) ?? metrics.find((m) => !campaign.metrics.some((cm
      ) => cm.name === m));
  if (bad || groupBy.length > 2 || !metrics.length) throw Object.assign(new Error(`Unsupported query field $
      {bad ?? ''}`.trim()), { status: 400 });
```

```
  const n = BigInt('0x' + campaign.n);
  const n2 = n * n;
  const width = cipherWidth(campaign.key_bits);
  const columns = cipherColumns(campaign.metrics);
  const wanted = columns.map((c, i) => [c, i]).filter(([c]) => metrics.includes(c.metric));

  // ponytail: loads the whole campaign in memory (~13 KB/record at 2048-bit);
  // stream by seq range and fan out to worker_threads past ~100k records.
  let rows = await all(
    `SELECT seq, ${DIMENSIONS.join(', ')}, cipher, leaf_hash FROM records WHERE campaign_id = ? ORDER BY seq
       `,
    [campaign.id],
  );
  if (fault.mode === 'drop') rows = rows.slice(1);

  const groups = new Map();
  for (const row of rows) {
    const key = Object.fromEntries(groupBy.map((d) => [d, row[d]]));
    const id = JSON.stringify(key);
    if (!groups.has(id)) groups.set(id, { key, cols: wanted.map(() => []) });
    const cells = splitCipher(row.cipher, width);
    const g = groups.get(id);
    wanted.forEach(([, i], j) => g.cols[j].push(cells[i]));
  }

  if (fault.mode === 'tamper' && rows.length) {
    const first = groups.values().next().value;
    first.cols[0][0] = (first.cols[0][0] * first.cols[0][0]) % n2;
  }

  const released = [];
  let suppressed = 0;
  for (const g of [...groups.values()].sort((a, b) => JSON.stringify(a.key).localeCompare(JSON.stringify(b.k
      ey)))) {
    const count = g.cols[0].length;
    if (count < K_MIN) { suppressed++; continue; }
    const sums = {};
    wanted.forEach(([c], j) => { sums[c.squared ? `${c.metric}^2` : c.metric] = aggregate(g.cols[j], n2).toS
      tring(16); });
    released.push({ key: g.key, n: count, sums });
  }

  return {
    campaign: campaign.id,
    // Root and size of the input set actually aggregated; the owner compares
    // these with its receipt, so a dropped or extra record is caught here.
    root: merkleRoot(rows.map((r) => r.leaf_hash)),
    total: rows.length,
    group_by: groupBy,
    metrics: wanted.map(([c]) => (c.squared ? `${c.metric}^2` : c.metric)),
    k_min: K_MIN,
    suppressed,
    groups: released,
  };
}
```

Listing 9. cloud/src/board.js: appending a signed, chained entry

```
let tail = Promise.resolve();

export function append(kind, campaignId, body) {
  const run = tail.then(async () => {
    const last = await one('SELECT seq, hash FROM board ORDER BY seq DESC LIMIT 1');
    const entry = {
      seq: (last?.seq ?? 0) + 1,
      kind,
      prev_hash: last?.hash ?? GENESIS,
      payload: JSON.stringify({ ...body, at: new Date().toISOString() }),
    };
    entry.hash = entryHash(entry);
    entry.sig = signHash(entry.hash, signingKey);
    await pool.query(
      'INSERT INTO board (seq, kind, campaign_id, payload, prev_hash, hash, sig) VALUES (?, ?, ?, ?, ?, ?, ?
        )',
      [entry.seq, kind, campaignId, entry.payload, entry.prev_hash, entry.hash, entry.sig],
    );
    return { ...entry, campaign_id: campaignId };
  });
  tail = run.catch(() => {});
  return run;
}
```

Listing 10. cloud/src/auth.js: role policy middleware with request logging

```
// allow(...roles) authenticates the bearer token and enforces the role policy.
export const allow = (...roles) => async (req, res, next) => {
  const started = Date.now();
  const action = `${req.method} ${req.route?.path ?? req.path}`;
  const token = req.get('authorization')?.replace(/^Bearer /, '');
  const user = token && await one(
    'SELECT u.username, u.role FROM sessions s JOIN users u ON u.id = s.user_id WHERE s.token = ? AND s.expi ↵
        res_at > NOW()',
    [token],
  );

  if (!user || !roles.includes(user.role)) {
    await logRequest({ username: user?.username, role: user?.role, action, outcome: 'denied', detail: user ? ↵
         'role not permitted' : 'no valid session' });
    return res.status(user ? 403 : 401).json({ error: user ? 'Your role is not allowed to do this' : 'Please ↵
         sign in' });
  }

  req.user = user;
  req.token = token;
  res.on('finish', () => logRequest({
    username: user.username,
    role: user.role,
    action,
    campaignId: req.params.id ? Number(req.params.id) : null,
    detail: res.locals.detail,
    records: res.locals.records,
    ms: Date.now() - started,
    outcome: res.statusCode < 400 ? 'allowed' : res.statusCode < 500 ? 'denied' : 'error',
  }));
  next();
};
```

Listing 11. cloud/src/index.js: ingest and commit endpoints

```
app.post('/campaigns/:id/records', allow('owner'), campaign('uploading'), async (req, res) => {
  const c = req.campaign;
  const expected = cipherColumns(c.metrics).length * cipherWidth(c.key_bits);
  const records = req.body?.records;
  if (!Array.isArray(records) || !records.length || records.length > 5000) return res.status(400).json({ err ↵
       or: 'Send 1-5000 records per batch' });

  const rows = [];
  for (const rec of records) {
    const cipher = Buffer.from(String(rec.cipher ?? ''), 'base64');
    const bad = !Number.isInteger(rec.seq) || rec.seq < 0 || cipher.length !== expected
      || DIMENSIONS.some((d) => typeof rec.dims?.[d] !== 'string' || rec.dims[d].length > 32);
    if (bad) return res.status(400).json({ error: `Malformed record ${rec.seq}` });
    rows.push([c.id, rec.seq, ...DIMENSIONS.map((d) => rec.dims[d]), cipher, leafHash(rec.dims, cipher)]);
  }
  await pool.query(`INSERT INTO records (campaign_id, seq, ${DIMENSIONS.join(', ')}, cipher, leaf_hash) VALU ↵
       ES ?`, [rows]);
  res.locals.records = rows.length;
  res.json({ leaves: rows.map((r) => r[r.length - 1]) });
});

// Seals the input set: the signed Merkle root is the owner's receipt.
app.post('/campaigns/:id/commit', allow('owner'), campaign('uploading'), async (req, res) => {
  const c = req.campaign;
  const leaves = (await all('SELECT leaf_hash FROM records WHERE campaign_id = ? ORDER BY seq', [c.id])).map ↵
       ((r) => r.leaf_hash);
  const root = merkleRoot(leaves);
  await pool.query("UPDATE campaigns SET status = 'committed', merkle_root = ?, record_count = ? WHERE id =  ↵
       ?", [root, leaves.length, c.id]);
  const entry = await board.append('batch_committed', c.id, { campaign: c.id, root, count: leaves.length });
  res.locals.records = leaves.length;
  res.json({ root, count: leaves.length, entry });
});
```

6.4 Auditor and frontend

Listing 12. web/src/audit.worker.js: input set, computation and decryption checks

```
    // 2. input set: rebuild the Merkle root from what the cloud publishes
    const records = await allRecords(id);
    const width = cipherWidth(bits);
    const cols = cipherColumns(metrics);
    const blobs = records.map((r) => fromBase64(r.cipher));
    const root = merkleRoot(records.map((r, i) => leafHash(r.dims, blobs[i])));
    inputs++;
    if (root !== commit.body.root || records.length !== commit.body.count) {
      inputFail.push(`campaign #${id}: published ciphertexts give root ${root.slice(0, 12)}…, receipt #${com ↵
        mit.seq} says ${commit.body.root.slice(0, 12)}…`);
    }

    const cells = blobs.map((b) => splitCipher(b, width));
    const queries = parsed.filter((e) => e.kind === 'query_result' && e.campaign_id === id);
    for (const q of queries) {
      const body = q.body;
      if (body.root !== commit.body.root || body.total !== commit.body.count) {
        inputFail.push(`query #${q.seq}: aggregated ${body.total} records under root ${body.root.slice(0, 12 ↵
        )}…, not the committed set`);
      }
      // 3. computation: recompute every released product from public ciphertexts
      say({ type: 'progress', stage: 'compute', text: `query #${q.seq}: recomputing ${body.groups.length * b ↵
        ody.metrics.length} sums` });
      for (const grp of body.groups) {
        const members = records.map((r, i) => i).filter((i) => body.group_by.every((d) => records[i].dims[d] ↵
         === grp.key[d]));
        if (members.length !== grp.n) computeFail.push(`query #${q.seq} ${JSON.stringify(grp.key)}: n=${grp. ↵
        n} but ${members.length} records match`);
        for (const col of body.metrics) {
          const [metric, sq] = col.split('^');
          const idx = cols.findIndex((c) => c.metric === metric && c.squared === Boolean(sq));
          const C = aggregate(members.map((i) => cells[i][idx]), n2);
          sums++;
          if (C.toString(16) !== grp.sums[col]) computeFail.push(`query #${q.seq} ${Object.values(grp.key).j ↵
        oin(' · ') || 'all'} / ${col}: product of ciphertexts ≠ published C`);
        }
      }

      // 4. decryption: g^m · r^n ≡ C (mod n²) for every published plaintext
      for (const d of parsed.filter((e) => e.kind === 'decryption_published' && e.body.query_seq === q.seq)) ↵
         {
        for (const [k, v] of d.body.values.entries()) {
          const C = BigInt('0x' + body.groups[v.group].sums[v.column]);
          proofs++;
          if (!verifyDecryption({ n, g, C, m: BigInt('0x' + v.m), r: BigInt('0x' + v.r) })) {
            decryptFail.push(`proof #${d.seq} value ${k} (${v.column}): g^m·r^n does not equal C`);
          }
          if (proofs % 25 === 0) say({ type: 'progress', stage: 'decrypt', text: `${proofs} decryption proof ↵
        s checked` });
        }
      }
    }
  }

  report('input', inputFail, inputs, `${inputs} campaign(s) rebuilt from published ciphertexts`);
  report('compute', computeFail, sums, `${sums.toLocaleString()} encrypted sums recomputed`);
  report('decrypt', decryptFail, proofs, `${proofs.toLocaleString()} published plaintexts checked`);
```

Listing 13. web/src/router.js: routes and role guard

```
const routes = [
  { path: '/', redirect: () => (sessions.owner ? '/campaigns' : sessions.cloud?.role === 'admin' ? '/monitor ↵
        ' : sessions.cloud ? '/board' : '/login') },
  { path: '/login', component: () => import('./views/Login.vue'), meta: { bare: true } },
  { path: '/campaigns', component: () => import('./views/Campaigns.vue'), meta: { domain: 'owner', title: 'C ↵
       ampaigns' } },
  { path: '/analysis/:id?', component: () => import('./views/Analysis.vue'), meta: { domain: 'owner', title: ↵
        'Analysis' } },
  { path: '/data/:id?', component: () => import('./views/Data.vue'), meta: { domain: 'owner', title: 'Source ↵
       data' } },
  { path: '/receipts/:id?', component: () => import('./views/Receipts.vue'), meta: { domain: 'owner', title: ↵
       'Receipts' } },
  { path: '/board', component: () => import('./views/Board.vue'), meta: { domain: 'cloud', roles: ['auditor' ↵
       , 'admin'], title: 'Bulletin board' } },
  { path: '/database/:table?', component: () => import('./views/Database.vue'), meta: { domain: 'cloud', rol ↵
       es: ['admin'], title: 'Database' } },
```

```
  { path: '/monitor', component: () => import('./views/Monitor.vue'), meta: { domain: 'cloud', roles: ['admi ↵
        n'], title: 'Monitor' } },
];

const router = createRouter({ history: createWebHistory(), routes });
```

7 User interface design

The interface is built around one idea: the user should always know which side of the trust boundary they are looking at. The navigation rail is painted in two halves, a warm paper tone for the owner above a dashed boundary marker and a cool slate for the cloud below it. The page background follows the domain of the current page, and the sign in screen is split down the middle with one form per domain.

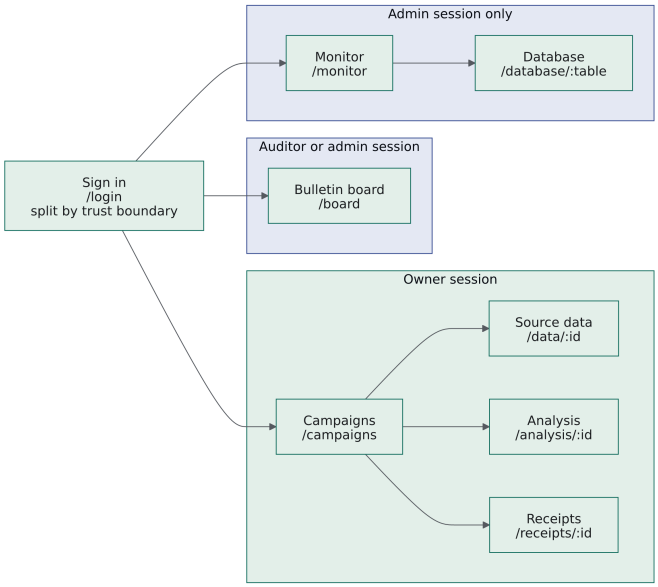


Figure 9. Pages and the session each one requires. The router sends a user without the right role back to the sign in page, and the navigation rail lists only the cloud pages the signed in role may open.

7.1 Charts

Charts are drawn with Apache ECharts 5, imported module by module, which keeps only the bar, line, scatter, heatmap and custom chart types in the build. One small Vue component wraps ECharts, renders to SVG and redraws on resize. Every chart is built in the browser from the decrypted sums the owner agent returns; the cloud never produces a chart or sees the numbers behind one.

7.2 Pages

Page	Role	Purpose
Sign in	anyone	Two forms side by side, one per domain.
Campaigns	owner	Choose the records, key length and measures; see an estimate of ciphertext count, upload size and time; follow the job phase by phase.
Source data	owner	The campaign's rows in plaintext, with each column marked encrypted, sent in clear or kept by the owner. Exists only on the owner side.
Analysis	owner	Five grouped queries, each with its five checks, followed by charts and an accuracy ledger comparing every decrypted sum with the plaintext sum.
Receipts	owner	The signed receipt and an inclusion proof for any single record.
Bulletin board	auditor, admin	The four public checks and the full timeline of entries.
Monitor	admin	Traffic, latency per endpoint, ciphertext storage and the fault injection switch.
Database	admin	Read only view of the cloud tables. Password hashes and session tokens are never selected.

8 Runtime screen captures

The captures below were taken from the running system under Docker Compose with the synthetic test dataset.

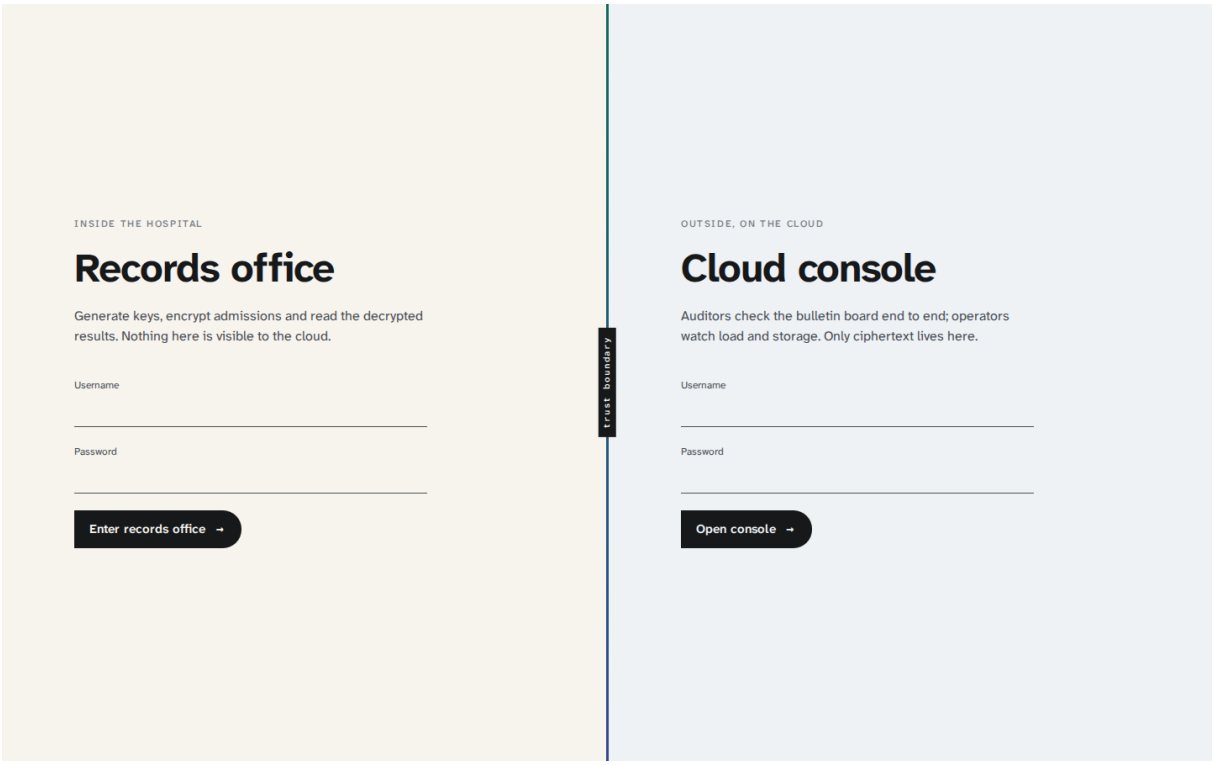


Figure 10. Sign in. The owner form on the left and the cloud console on the right, divided by the trust boundary.

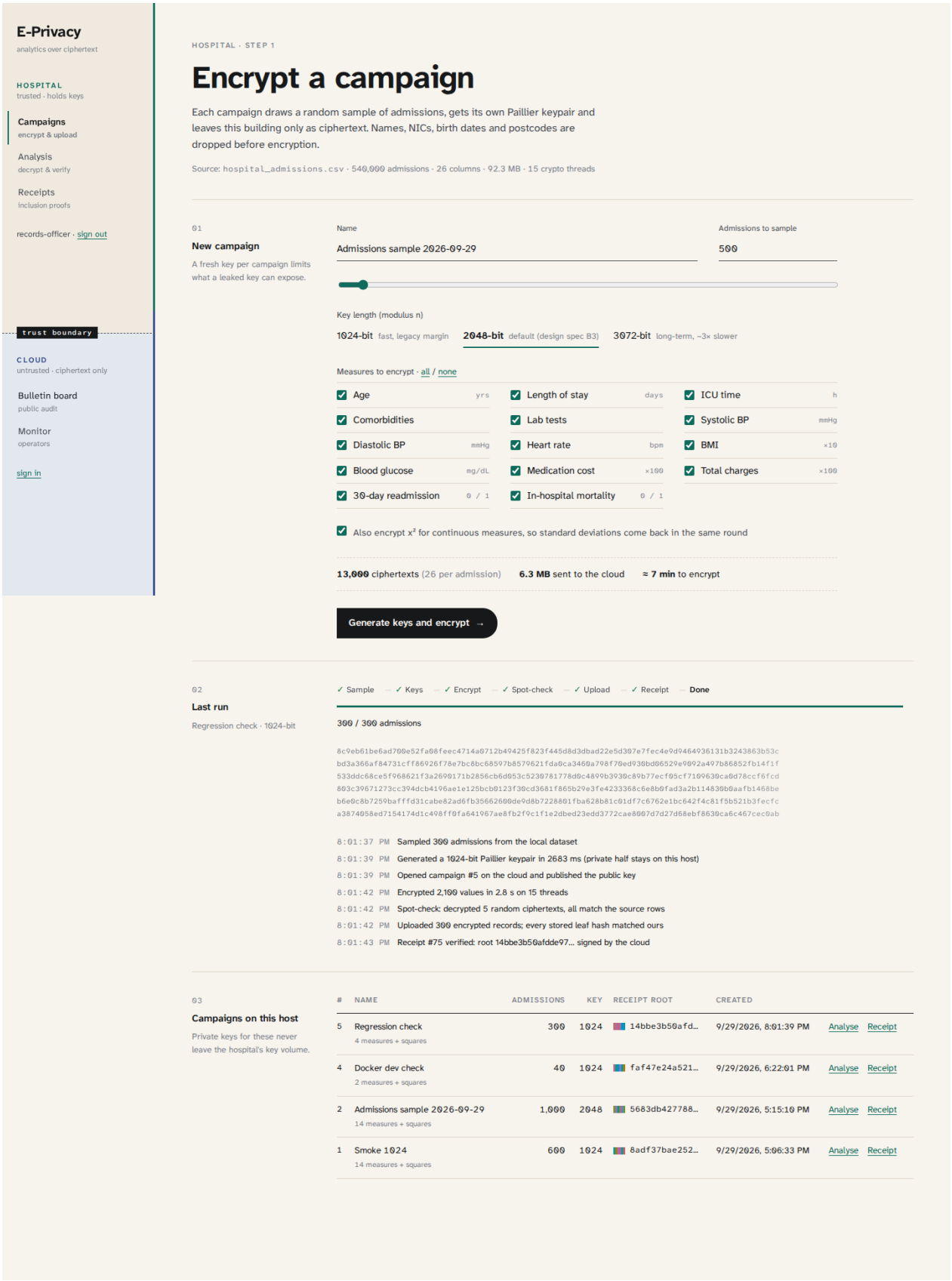


Figure 11. Campaigns page. The form shows the ciphertext count, upload size and estimated time before anything starts. Below it, the last run with its phase list, the first ciphertexts and a timestamped log of every step.

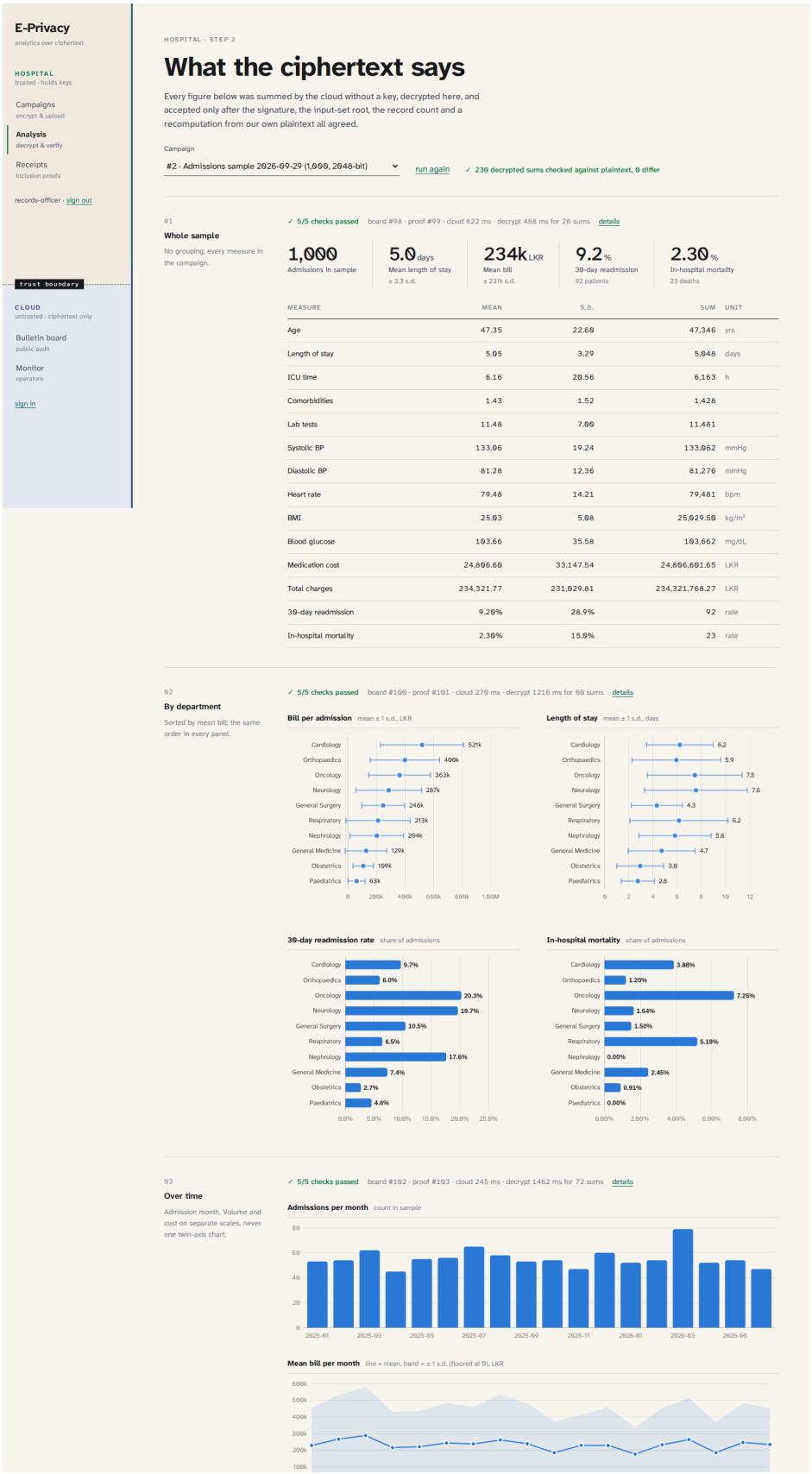


Figure 12. Analysis, first section. All five checks passed; headline figures and the overall table are decrypted from sums the cloud computed without the key.

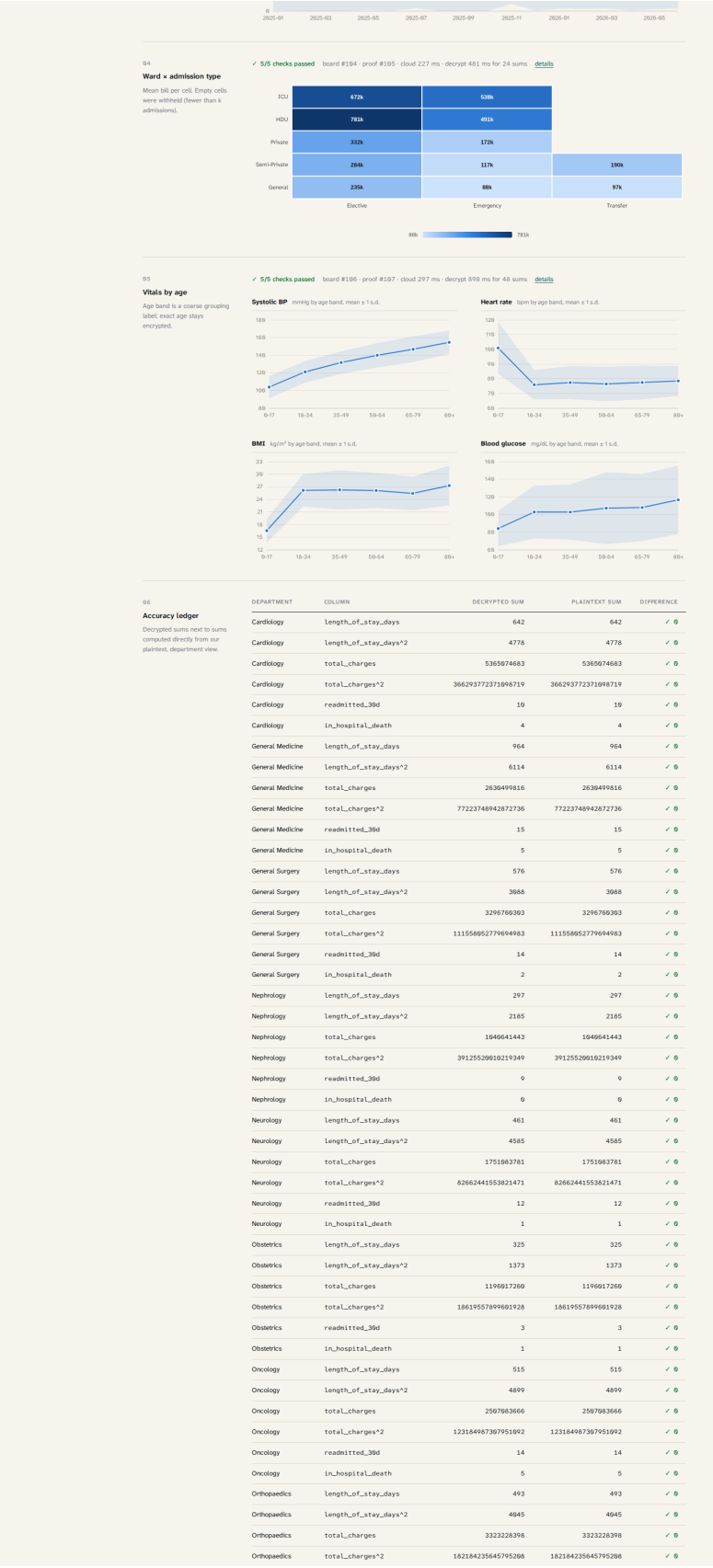


Figure 13. Analysis, grouped views. Means with spread per group, a monthly trend on two stacked charts and a heatmap where withheld groups stay empty.

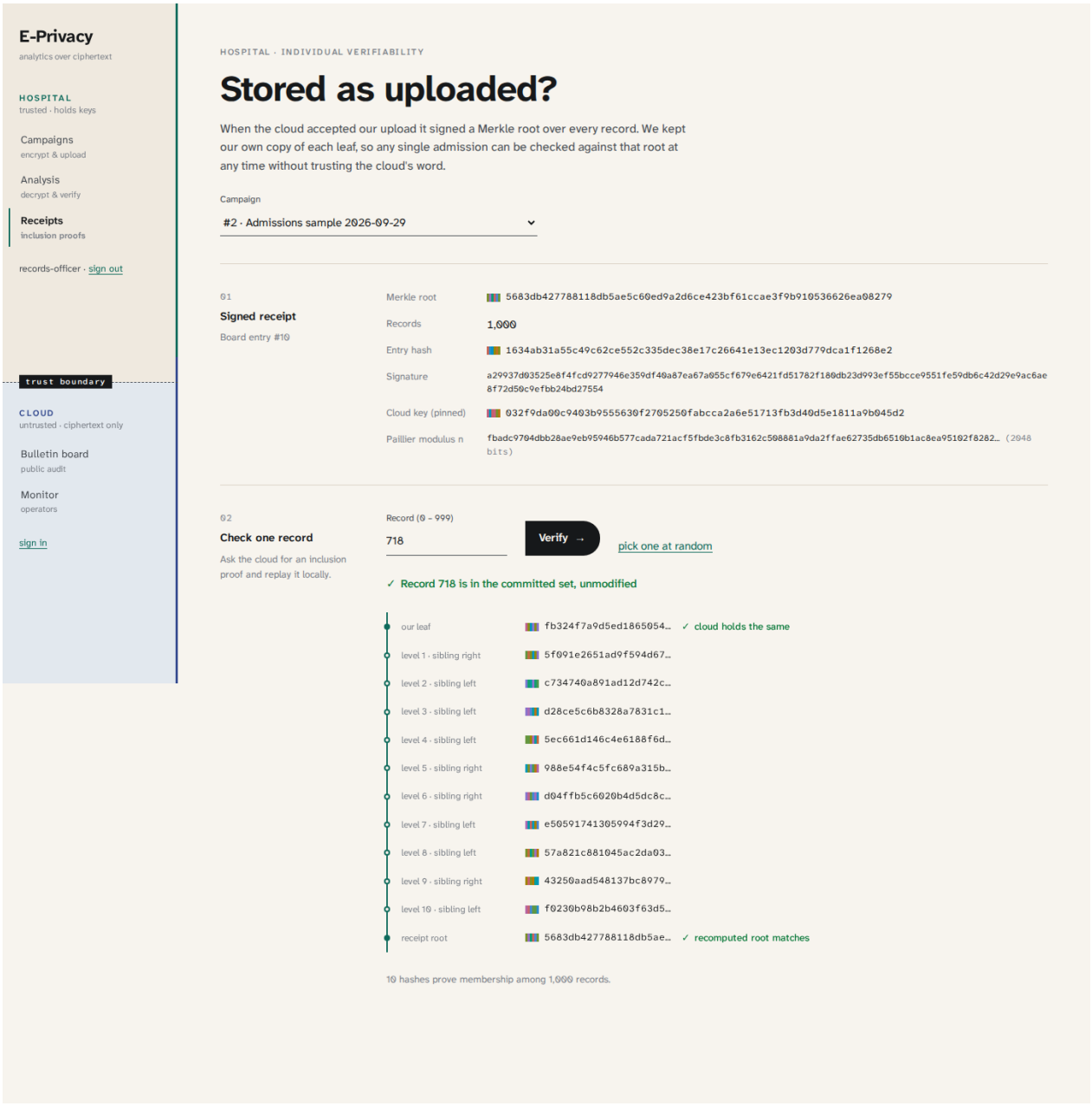


Figure 14. Receipts. The signed receipt and a verified inclusion proof for record 718: ten sibling hashes prove membership among 1,000 records.

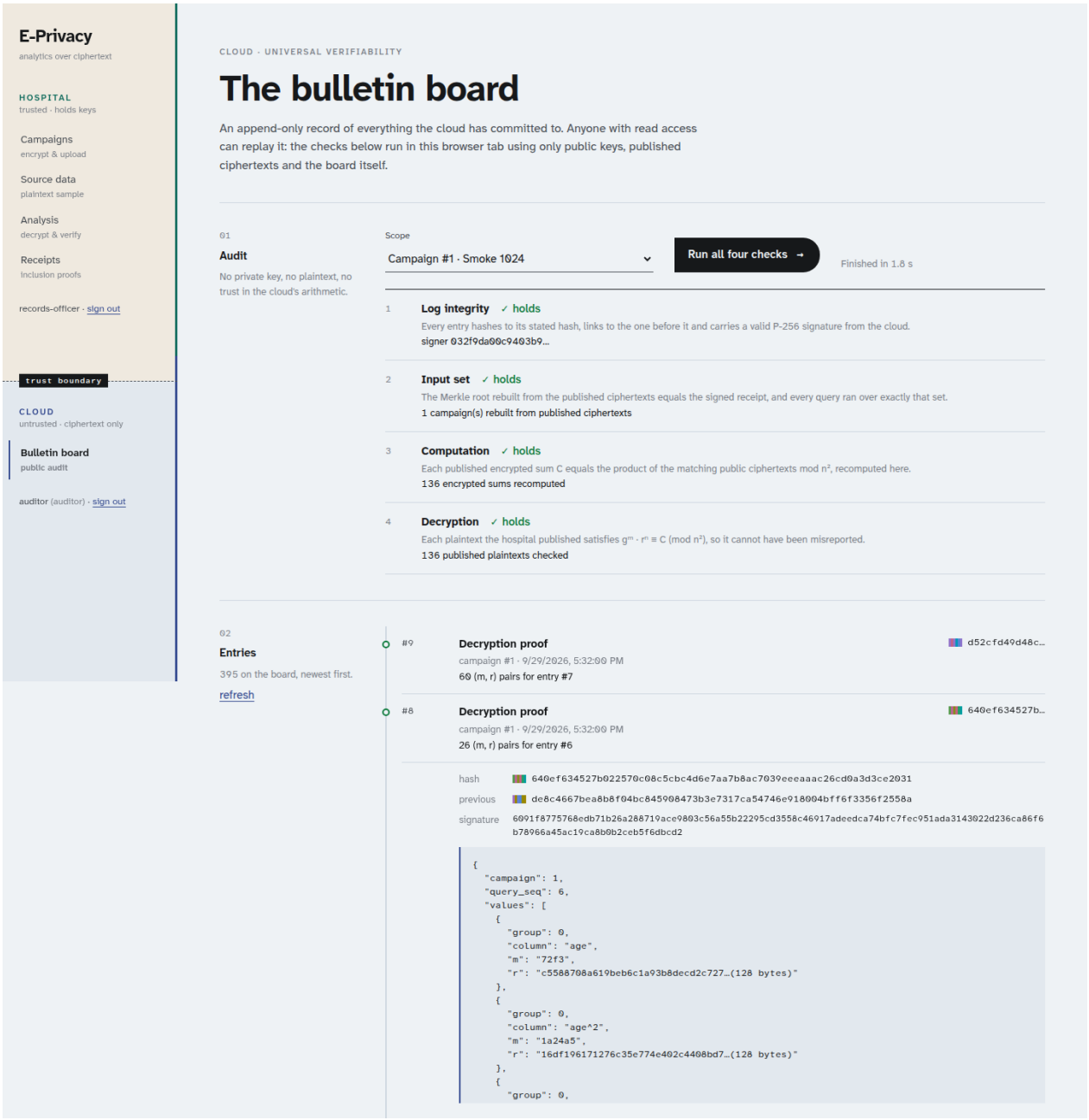


Figure 15. Bulletin board after an honest run. All four checks hold; one entry is expanded to show its hash, link, signature and payload.

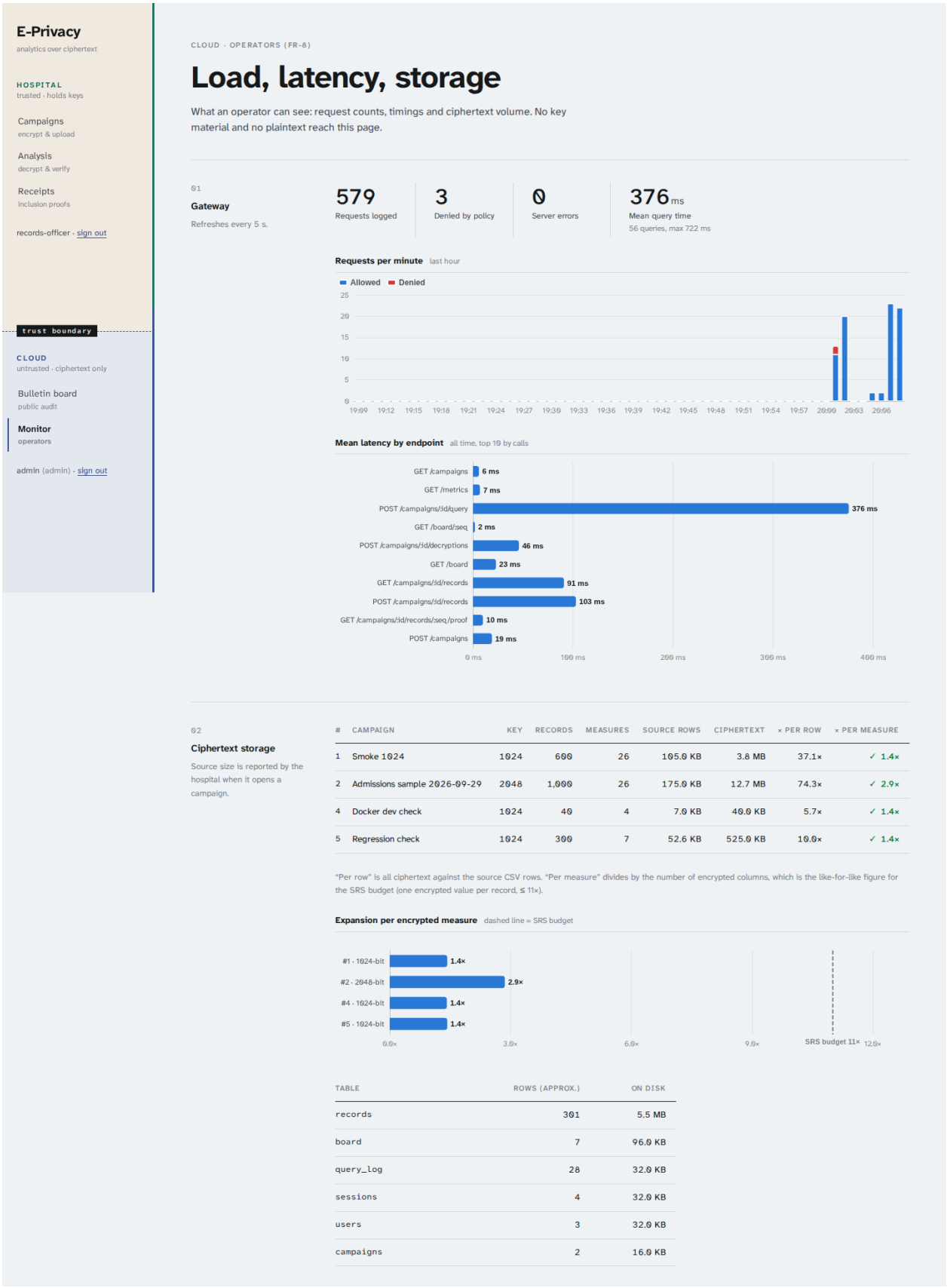


Figure 16. Monitor. Requests per minute, mean latency per endpoint and ciphertext storage per campaign. No key material or plaintext reaches this page.

9 Testing and measured results

9.1 Protocol unit tests

The shared protocol has four tests that use a real 1024 bit key. They run with *npm test* and all four pass.

Test	Result	Time
Aggregate over ciphertext decrypts to the plaintext sum, and the decryption proof holds	pass	985 ms
Merkle proofs verify for every leaf and fail after tampering	pass	4.7 ms
Leaf hash binds dimensions to ciphertext	pass	0.3 ms
Board chain detects edits and forged signatures	pass	64 ms

9.2 Fault injection

The operator can switch the cloud into a cheating mode. *Drop* leaves one record out of every query and *tamper* squares one ciphertext before summing. Both were run against a committed campaign. In each case the owner refused the result and the auditor's matching check failed, while the remaining checks still held.

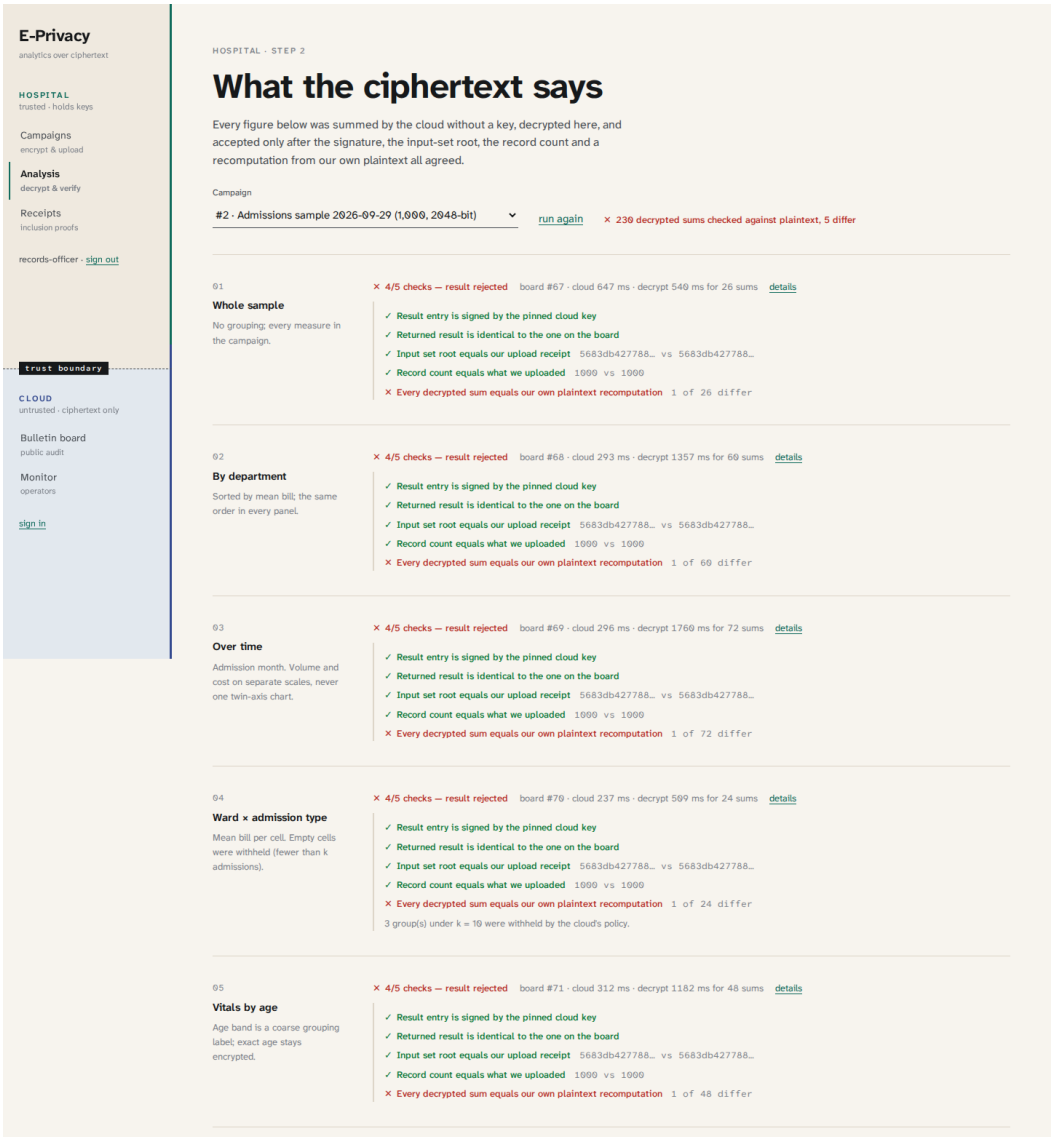


Figure 17. Analysis with the tamper fault on. Four of five checks pass, but one decrypted sum differs from the plaintext recomputation and the result is rejected.

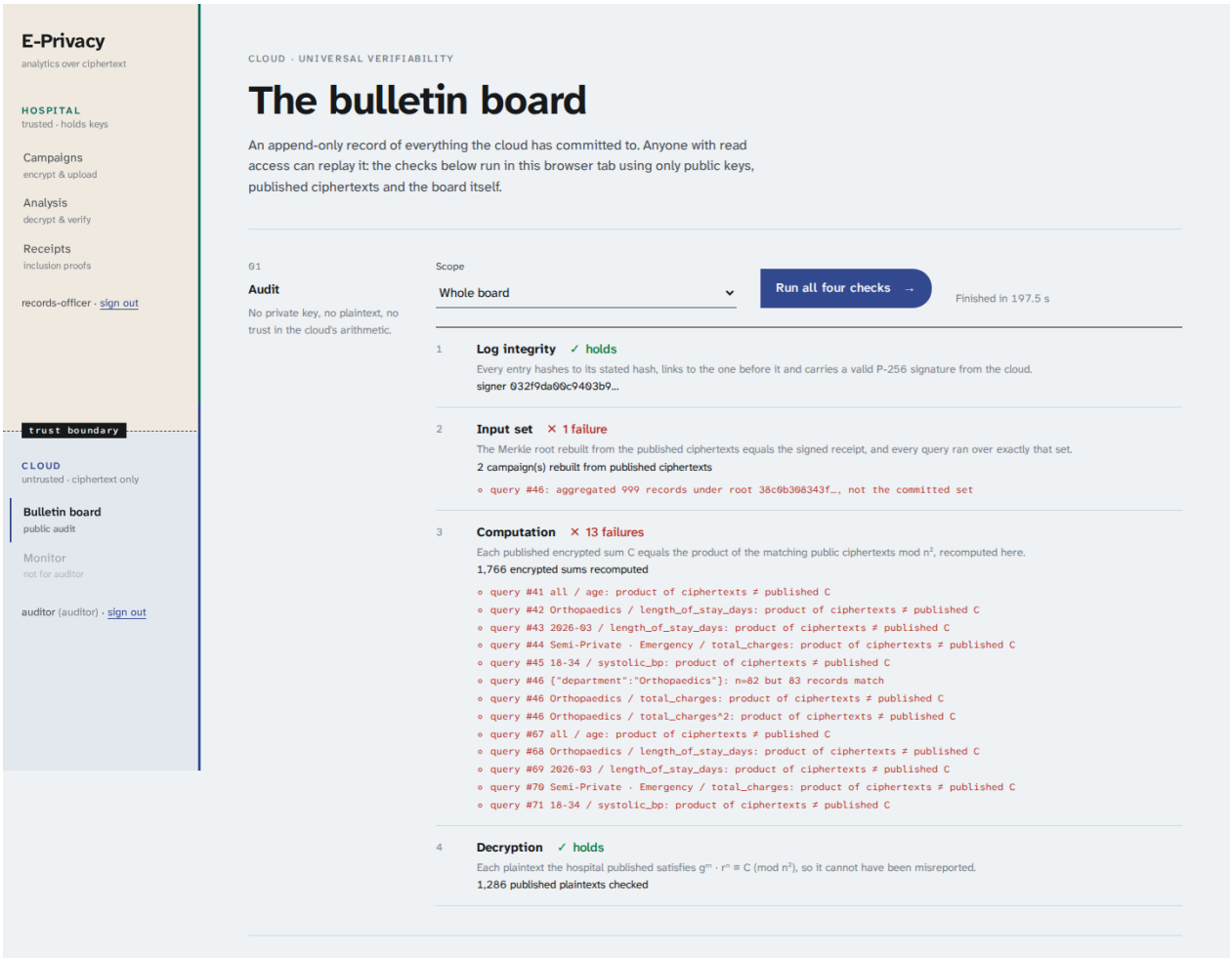


Figure 18. Audit after faulty queries. Log integrity holds, while the input set and computation checks list the exact entries that do not match.

9.3 Performance

Measured on an 8 core laptop (AMD Ryzen 7 7730U) under Docker Desktop.

Operation	1024 bit	2048 bit
Key generation	0.4 to 0.6 s	1.0 to 1.6 s
Encrypt one value	7 ms	43 to 45 ms
Decrypt one value	5 ms	30 ms
Homomorphic addition (one multiply mod n^2)	4 µs	14 µs

Measurement	Value
Campaign of 600 records, 26 values each, 1024 bit	22.4 s to encrypt
Campaign of 1,000 records, 26 values each, 2048 bit	1,022 s to encrypt, under 3 s to upload
Cloud query over 1,000 records at 2048 bit	383 ms mean, 720 ms max
Commit: Merkle root over 1,000 leaves and signature	22 ms
Full audit: 45 entries, 1,600 records, 1,056 sums, 826 proofs	118.7 s in one Web Worker
Storage per encrypted measure (2048 bit)	2.9 times the source bytes

Homomorphic addition is about 3,000 times cheaper than encryption. The owner side is the bottleneck, and it scales nearly linearly with physical cores on the host.

10 Limitations and future work

- The cloud holds one campaign in memory while it aggregates. This suits tens of thousands of records; beyond that it should stream by sequence range and fan out to worker threads.
- The owner pins the cloud's signing key the first time it connects (trust on first use).
- Grouping labels travel in clear. Small groups are withheld, but the labels remain quasi identifiers.
- Only additive statistics are possible: counts, sums, means, rates and standard deviations. Medians and other order statistics need a different protocol.
- Planned next: differential privacy noise on released results, and studies across several data owners with threshold decryption.